API Flask/FastAPI

m2iformation.fr

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

API Flask/FastAPI

Création d'APIs REST Modernes

Utopios® Tous droits réservés

Module 2 - FastAPI Avancé + Validation

Maîtrisez le framework moderne pour les APIs

Pydantic, validation, dépendances, async, performance et caching

Fondamentaux | Validation | Performance

Utopios® Tous droits réservés

Qu'est-ce que FastAPI?

Le framework web moderne et rapide

FastAPI est un framework Python moderne pour construire des APIs REST avec validation automatique et documentation interactive.

- Créé en 2018 par Sebastián Ramírez
- Basé sur Starlette pour l'async et Pydantic pour la validation
- Très rapide: performances au niveau de Node.js et Go

Utopios® Tous droits réservés

Qu'est-ce que FastAPI?

Le framework web moderne et rapide

- Documentation automatique avec Swagger et ReDoc
- Support natif de async/await
- Validation des données automatique

Avantage principal: FastAPI génère automatiquement la documentation OpenAPI et valide toutes les données entrantes!

Utopios® Tous droits réservés

Installation et Configuration

Mettre en place FastAPI

Installez les dépendances essentielles:

```
pip install fastapi uvicorn python-multipart
```

Utopios® Tous droits réservés

Installation et Configuration

Créez votre première application:

```python
from fastapi import FastAPI

app = FastAPI(title="Mon API", version="1.0.0")

@app.get("/")
def read_root():
    return {"message": "Hello, FastAPI!"}

@app.get("/api/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}
```

Lancez avec Uvicorn:

```
uvicorn main:app --reload
```

Visitez http://localhost:8000/docs pour la documentation interactive!

Utopios® Tous droits réservés

Routes Décorées

Les décorateurs HTTP de FastAPI

FastAPI propose des décorateurs pour chaque méthode HTTP:

```python
@app.get("/api/users")
async def list_users():
    return []

@app.post("/api/users")
async def create_user(user_data: dict):
    return {"id": 1, "created": True}

@app.get("/api/users/{user_id}")
async def get_user(user_id: int):
    return {"id": user_id}

@app.put("/api/users/{user_id}")
async def update_user(user_id: int, updates: dict):
    return {"id": user_id, "updated": True}

@app.delete("/api/users/{user_id}")
async def delete_user(user_id: int):
    return {"deleted": True}
```

Async par défaut: FastAPI encourage l'utilisation d'async/await pour de meilleures performances!

Utopios® Tous droits réservés

Modèles Pydantic

Validation automatique des données

Pydantic utilise les type hints Python pour valider les données:

```python
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str
    email: str
    age: int | None = None

class UserCreate(BaseModel):
    name: str
    email: str
    age: int

@app.post("/api/users")
async def create_user(user: UserCreate):
    return {"id": 1, **user.model_dump()}
```

Utopios® Tous droits réservés

Modèles Pydantic

Validation automatique des données

- Validation automatique des types
- Conversion automatique (str → int)
- Erreurs claires en JSON
- Support des types optionnels (| None)

Utopios® Tous droits réservés

Type Hints et Validation

FastAPI valide automatiquement

Les type hints Python activent la validation automatique:

```python
from typing import Optional

@app.get("/search")
async def search(
    query: str,           # Obligatoire
    limit: int = 10,      # Défaut 10
    skip: int = 0,        # Défaut 0
    active: bool = True   # Défaut True
):
    return {"query": query, "limit": limit}

@app.post("/users")
async def create(user: UserCreate):
    # 'user' est validé automatiquement
    # Si invalide, FastAPI retourne 422 automatiquement
    return user
```

Avantage: Pas besoin de vérifier manuellement les types ou valeurs!

Utopios® Tous droits réservés

Documentation Automatique

Swagger UI et ReDoc généré automatiquement

FastAPI génère automatiquement la documentation complète:

- **Swagger UI:** http://localhost:8000/docs - Interface interactive
- **ReDoc:** http://localhost:8000/redoc - Documentation épurée
- **OpenAPI:** http://localhost:8000/openapi.json - Spécification complète

Utopios® Tous droits réservés

Documentation Automatique

Personnalisez la documentation:

```
app = FastAPI(
    title="API Utilisateurs",
    description="API pour gérer les utilisateurs",
    version="1.0.0",
    openapi_url="/api/v1/openapi.json"
)
```

Documentation vivante: Testez directement depuis Swagger!

Utopios® Tous droits réservés

Contraintes de Champs (Field Constraints)

Validation poussée avec Pydantic Field

Définissez des contraintes détaillées sur chaque champ:

```
from pydantic import Field

class User(BaseModel):
    name: str = Field(..., min_length=1, max_length=100)
    email: str = Field(..., pattern=r"^[\w\.-]+@[\w\.-]+\.\w+$")
    age: int = Field(..., gt=0, lt=150)
    phone: str | None = Field(None, regex=r"^\+?1?\d{9,15}$")
    salary: float = Field(default=0, ge=0)

class Product(BaseModel):
    name: str = Field(..., description="Nom du produit")
    price: float = Field(..., gt=0, description="Prix > 0")
    quantity: int = Field(default=0, ge=0, description="Stock")
    discount: float = Field(default=0, ge=0, le=100)
```

Utopios® Tous droits réservés

Contraintes de Champs (Field Constraints)

Les contraintes disponibles:

- **gt, lt, ge, le:** Comparaisons numériques
- **min_length, max_length:** Longueur de chaîne
- **pattern, regex:** Expressions régulières

Utopios® Tous droits réservés

Validateurs Personnalisés

Logique de validation métier

Créez des validateurs personnalisés avec @field_validator:

```
from pydantic import BaseModel, field_validator

class User(BaseModel):
    name: str
    email: str
    age: int

    @field_validator('name')
    @classmethod
    def name_not_empty(cls, v):
        if not v or v.isspace():
            raise ValueError('Le nom ne peut pas être vide')
        return v.strip()

    @field_validator('email')
    @classmethod
    def email_valid(cls, v):
        if '@' not in v or '.' not in v:
            raise ValueError('Email invalide')
        return v.lower()

    @field_validator('age')
    @classmethod
    def age_valid(cls, v):
        if v < 18:
            raise ValueError('Age minimum: 18 ans')
        return v
```

Utopios® Tous droits réservés

Modèles Imbriqués (Relationships)

Structures complexes et anidées

Définissez des relations entre modèles:

```python
from pydantic import BaseModel

class Address(BaseModel):
    street: str
    city: str
    zipcode: str
    country: str

class User(BaseModel):
    name: str
    email: str
    address: Address  # Modèle imbriqué

class Company(BaseModel):
    name: str
    employees: list[User]  # Liste de modèles
    headquarters: Address

# Utilisation
company_data = {
    "name": "TechCorp",
    "headquarters": {
        "street": "123 Tech St",
        "city": "Paris",
        "zipcode": "75000",
        "country": "France"
    },
    "employees": [...]
}
```

Utopios® Tous droits réservés

Listes et Dictionnaires

Collections typées en Pydantic

Pydantic valide les collections avec leurs éléments:

```python
from pydantic import BaseModel

class Product(BaseModel):
    name: str
    tags: list[str]  # Liste de chaînes
    prices: dict[str, float]  # Dict clé-valeur
    ratings: list[float]  # Liste de nombres

class Order(BaseModel):
    items: list[Product]  # Liste d'objets
    metadata: dict  # Dict sans contrainte
    notes: list[str] = []  # Défaut vide

# Validation automatique
order = Order(
    items=[...],
    metadata={"customer": "John"}
)
```

Type safety: Chaque élément est validé!

Utopios® Tous droits réservés

Union Types

Accepter plusieurs types

Utilisez Union pour accepter plusieurs types:

```python
from pydantic import BaseModel
from typing import Union

class Cat(BaseModel):
    name: str
    meow_volume: int

class Dog(BaseModel):
    name: str
    bark_volume: int

class Animal(BaseModel):
    pet: Union[Cat, Dog]  # Peut être Cat ou Dog

# Utilisation
cat = Animal(pet={"name": "Fluffy", "meow_volume": 5})
dog = Animal(pet={"name": "Rex", "bark_volume": 8})

@app.post("/animals")
async def create_animal(animal: Animal):
    return animal
```

Discrimination: Vous pouvez aussi utiliser des champs discriminants!

Utopios® Tous droits réservés

Configuration Avancée de Modèles

Contrôle complet du comportement Pydantic

Configurez le comportement des modèles:

```
from pydantic import BaseModel, ConfigDict

class User(BaseModel):
    model_config = ConfigDict(
        str_strip_whitespace=True,      # Strip les espaces
        validate_default=True,           # Valide les défauts
        arbitrary_types_allowed=True,    # Types arbitraires
        use_enum_values=True,            # Utilise valeurs enum
        json_schema_extra={              # Métadonnées OpenAPI
            "example": {
                "name": "John",
                "email": "john@example.com"
            }
        }
    )

    name: str
    email: str
    is_active: bool = True
```

Utopios® Tous droits réservés

Configuration Avancée de Modèles

Contrôle complet du comportement Pydantic

- **alias:** Noms d'attributs alternatifs
- **exclude:** Exclure des champs
- **frozen:** Rendre le modèle immutable

Utopios® Tous droits réservés

Dépendances (Dependency Injection)

Injecter des dépendances dans les routes

FastAPI supporte l'injection de dépendances:

```
from fastapi import Depends

def get_db():
    """Dépendance: connexion à la BD"""
    db = connect_to_database()
    try:
        yield db
    finally:
        db.close()

def get_current_user(token: str):
    """Dépendance: utilisateur courant"""
    user = verify_token(token)
    if not user:
        raise Exception("Utilisateur invalide")
    return user

@app.get("/api/users/{user_id}")
async def get_user(
    user_id: int,
    db = Depends(get_db),
    current_user = Depends(get_current_user)
):
    return db.query_user(user_id)
```

Avantage: Réutilisable, testable, lisible!

Utopios® Tous droits réservés

Scopes de Dépendances

Contrôler la durée de vie des dépendances

FastAPI propose plusieurs niveaux de scopes:

```
from fastapi import Depends
from uuid import uuid4

async def get_request_id():
    """Scope request: créé pour chaque requête"""
    return str(uuid4())

async def get_session():
    """Scope app: partagé par l'application"""
    return create_session()

@app.get("/")
async def read_root(
    request_id = Depends(get_request_id),
    session = Depends(get_session)
):
    return {
        "request_id": request_id,
        "session_id": session.id
    }
```

Utopios® Tous droits réservés

Scopes de Dépendances

Contrôler la durée de vie des dépendances

- **request:** Créé et détruit pour chaque requête
- **app:** Créé une fois et partagé
- **lifespan:** Cycle de vie complet (startup/shutdown)

Utopios® Tous droits réservés

Service Layer avec FastAPI

Séparation des couches

Implémentez le pattern Service Layer:

```
class UserService:
    def __init__(self, db):
        self.db = db

    async def create_user(self, user_data: UserCreate):
        user = User(**user_data.model_dump())
        self.db.add(user)
        await self.db.commit()
        return user

    async def get_user(self, user_id: int):
        return await self.db.query(User).filter(...).first()

async def get_user_service(db = Depends(get_db)):
    return UserService(db)

@app.post("/users")
async def create_user(
    user: UserCreate,
    service = Depends(get_user_service)
):
    return await service.create_user(user)
```

Utopios® Tous droits réservés

Repository Pattern

Abstraction de l'accès aux données

Implémentez le Repository pattern:

```python
class UserRepository:
    def __init__(self, db):
        self.db = db

    async def find_by_id(self, user_id: int):
        return await self.db.query(User).filter(...).first()

    async def find_all(self, skip: int = 0, limit: int = 10):
        return await self.db.query(User).offset(skip).limit(limit).all()

    async def create(self, user: UserCreate):
        db_user = User(**user.model_dump())
        self.db.add(db_user)
        await self.db.commit()
        await self.db.refresh(db_user)
        return db_user

    async def delete(self, user_id: int):
        user = await self.find_by_id(user_id)
        if user:
            await self.db.delete(user)
            await self.db.commit()
        return user
```

Utopios® Tous droits réservés

Async et Await

Programmation asynchrone en Python

FastAPI utilise async/await pour la performance:

```python
import asyncio

async def fetch_user(user_id: int):
    """Opération asynchrone"""
    await asyncio.sleep(1)  # Simule une BD
    return {"id": user_id, "name": f"User {user_id}"}

@app.get("/users/{user_id}")
async def get_user(user_id: int):
    user = await fetch_user(user_id)
    return user

@app.post("/process")
async def process(data: dict):
    # Faire plusieurs opérations en parallèle
    results = await asyncio.gather(
        fetch_user(1),
        fetch_user(2),
        fetch_user(3)
    )
    return {"results": results}
```

Avantage: Handle 1000+ requêtes simultanées!

Utopios® Tous droits réservés

Background Tasks

Exécuter des tâches en arrière-plan

FastAPI supporte les tâches en arrière-plan:

```python
from fastapi import BackgroundTasks
import time

def send_email(email: str, subject: str):
    """Tâche longue"""
    time.sleep(5)
    print(f"Email sent to {email}")

@app.post("/register")
async def register(user: UserCreate, background_tasks: BackgroundTasks):
    # Créer l'utilisateur
    new_user = await create_user(user)

    # Envoyer un email en arrière-plan
    background_tasks.add_task(
        send_email,
        email=user.email,
        subject="Welcome!"
    )

    return {"user": new_user, "status": "created"}
```

Non-bloquant: L'API retourne immédiatement!

Utopios® Tous droits réservés

Caching avec Redis

Cache pour les performances

Utilisez Redis pour cacher les données:

```
import redis
import json

cache = redis.Redis(host='localhost', port=6379, decode_responses=True)

async def get_users(skip: int = 0, limit: int = 10):
    # Clé de cache
    cache_key = f"users:list:{skip}:{limit}"

    # Vérifier le cache
    cached = cache.get(cache_key)
    if cached:
        return json.loads(cached)

    # Récupérer de la BD
    users = await db.query(User).offset(skip).limit(limit).all()

    # Cacher pour 3600 secondes
    cache.setex(cache_key, 3600, json.dumps(users))

    return users
```

Utopios® Tous droits réservés

Caching avec Redis

- **TTL:** Durée de vie du cache
- **Invalidation:** Supprimer le cache à jour
- **Warming:** Pré-remplir le cache

Utopios® Tous droits réservés

Performance et Optimisation 1/2

Rendre votre API ultra-rapide

- **Async/await:** Handle 1000+ requêtes par seconde
- **Connection pooling:** Réutiliser les connexions BD
- **Caching:** Redis, HTTP caching headers
- **Indexing:** Index sur les colonnes BD
- **Pagination:** Limiter la taille des réponses

Utopios® Tous droits réservés

Performance et Optimisation 2/2

- **Compression:** GZIP les réponses
- **Query optimization:** N+1 queries, eager loading

```
from fastapi.middleware.gzip import GZIPMiddleware

app.add_middleware(GZIPMiddleware, minimum_size=1000)
```

Benchmark: FastAPI atteint souvent 100k req/sec!

Utopios® Tous droits réservés

Comparaison: Flask vs FastAPI

Quand utiliser quoi?

Critère	Flask	FastAPI
Validation	Manuelle	Automatique
Documentation	Manuelle	Automatique
Async	Support	Natif
Performance	Bonne	Excellente
Apprentissage	Facile	Moyen

Conseil: FastAPI pour les APIs modernes, Flask pour la flexibilité

Utopios® Tous droits réservés

Démos Associées

Apprenez en pratiquant

Exécutez les démos FastAPI:

```
# Hello World FastAPI
uvicorn demos/demo3_fastapi_hello:app --reload

# CRUD complet
uvicorn demos/demo4_fastapi_crud:app --reload

# FastAPI + Sécurité
uvicorn demos/demo9_fastapi_security:app --reload
```

Utopios® Tous droits réservés

Ressources officielles:

- **Documentation FastAPI:** https://fastapi.tiangolo.com/
- **Pydantic docs:** https://docs.pydantic.dev/
- **Starlette:** https://www.starlette.io/

Utopios® Tous droits réservés

Résumé - Concepts Clés 1/2

- **FastAPI:** Framework moderne et rapide pour APIs
- **Pydantic:** Validation automatique des données
- **Type hints:** Sécurité des types en Python
- **Routes décorées:** @app.get, @app.post, etc

Utopios® Tous droits réservés

Résumé - Concepts Clés 2/2

- **Dépendances:** Injection de dépendances
- **Async/await:** Programmation asynchrone
- **Performance:** Cache, compression, optimisation
- **Patterns:** Service Layer, Repository Pattern

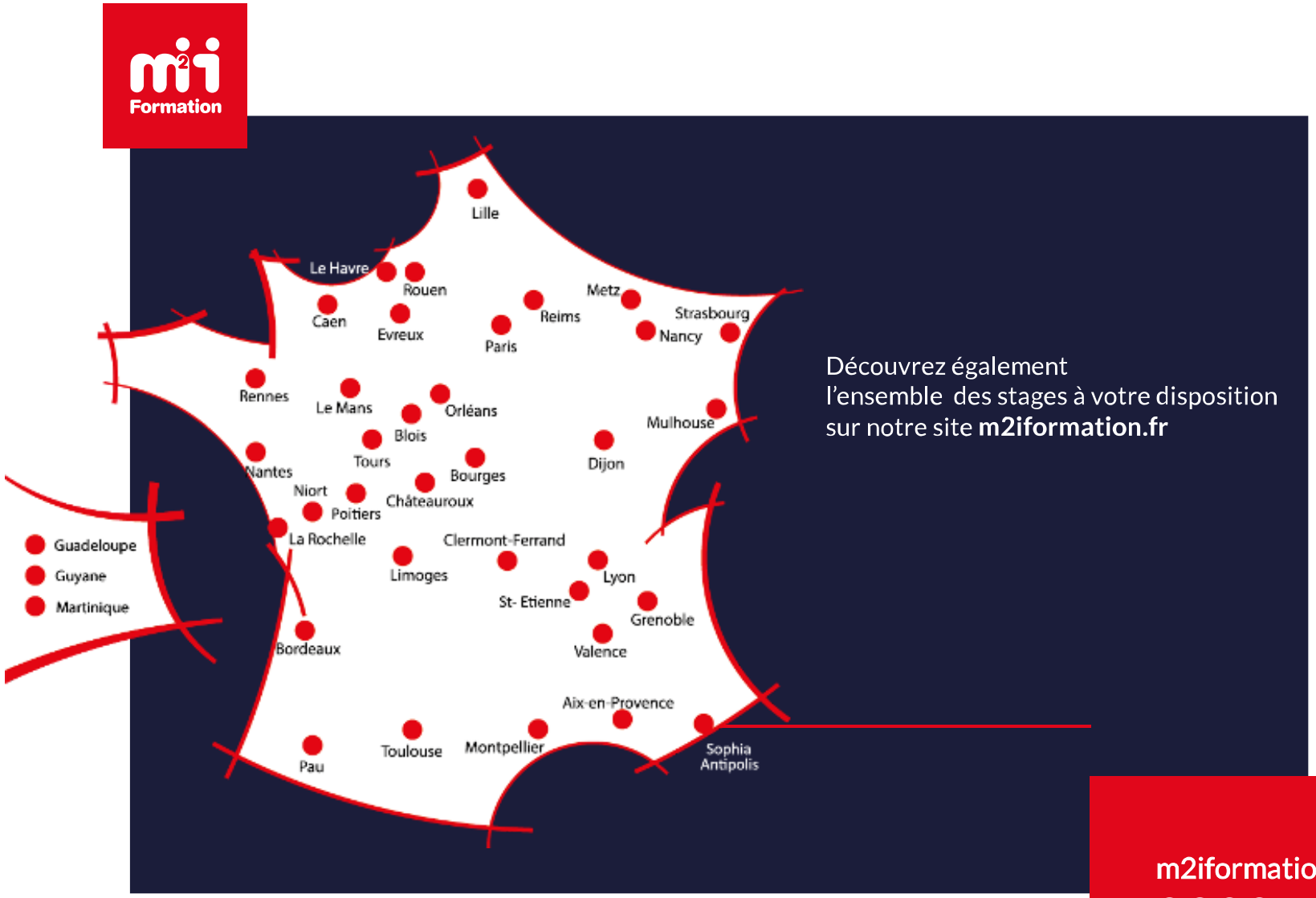
Utopios® Tous droits réservés

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Merci pour votre attention

Des questions ?

Utopios® Tous droits réservés



Découvrez également
l'ensemble des stages à votre disposition
sur notre site **m2information.fr**

m2iformation.fr